DIGITAL CREDENTIAL INFRASTRUCTURE

CertiChain

Privacy-aware on-chain credential verification

Project type Decentralized verification registry	**Platform** Ethereum-compatible EVM
Implementation Solidity 0.8.x	**Demo environment** Remix VM / local Ganache

A minimal decentralized registry that lets an institution publish a certificate fingerprint and lets any verifier check its issuance and current revocation status. Personal information and certificate files remain off-chain.

Engineering portfolio prototype

Date: October 2026

1. Project Overview

Academic certificates are commonly verified by contacting the issuing institution or consulting a central database. That process can be slow, difficult across institutions, and vulnerable to a single database being altered or unavailable. CertiChain demonstrates how an Ethereum smart contract can provide a shared, append-only record of certificate fingerprints and issuer actions.

The contract does not publish a certificate or personal details. The issuer hashes a canonical digital file off-chain and writes only the 32-byte digest, issuer wallet, timestamp, and revocation state. A verifier hashes the supplied file and queries the contract. A matching digest indicates that this exact file fingerprint was registered; it does not independently prove the academic claims in the document.

2. Product Context

Institutions and employers need a quick way to detect altered certificate files and confirm whether a record was issued and later revoked. A single institution-controlled database creates a central point of failure and makes cross-organization verification dependent on manual coordination. The project explores a shared ledger while keeping sensitive document content off-chain.

3. Design Goals

- Build an issuer-gated Solidity registry for certificate fingerprints.
- Allow authorized issuers to register a fingerprint once and emit an auditable event.
- Allow the issuing institution or registry owner to revoke a record.
- Expose a public read function returning existence, validity, issuer, and issue time.
- Demonstrate transaction state changes in an EVM-compatible local environment.

4. Technical Stack

Solidity: contract language, compiler target 0.8.x. **Ethereum Virtual Machine:** execution and persistent state model. **Remix IDE:** compile, deploy to Remix VM, and call contract methods. **Ganache:** optional local JSON-RPC development chain. **Browser UI:** responsive wallet-connected interface using ethers.js for contract calls and browser-native SHA-256 file hashing. The interface supports verification, issuance, and revocation; no public deployment is claimed.

5. System Architecture

Data flow: issuer canonicalizes the document, computes its digest, then submits a transaction. EVM nodes execute the contract and replicate the resulting state under the selected network consensus. A verifier independently computes the digest and makes a read-only query. A read call does not change chain state; issuance and revocation require signed transactions and gas.

6. Smart Contract Design

The source file is **contracts/CertiChain.sol**. Registry storage maps each bytes32 digest to issuer address, issue timestamp, revoked flag, and existence flag. A separate mapping controls authorized issuer wallets. The deployer is the owner and is authorized by the constructor.

Function / event	Purpose and access
`setIssuerAuthorization(address,bool)`	Owner-only. Grants or removes issuer permission; rejects the zero address.
`issueCertificate(bytes32)`	Authorized issuers only. Rejects empty and duplicate hashes; records issuer and timestamp; emits CertificateIssued.
`revokeCertificate(bytes32)`	Only original issuer or owner. Marks an existing record revoked once; emits CertificateRevoked.
`verifyCertificate(bytes32)`	Public view. Returns existence, current validity, issuer, and issue time.
`IssuerAuthorizationChanged`	Audit event when issuer permission changes.

Access control and trust assumptions

Authorization prevents arbitrary wallets from issuing records, but the owner remains a governance authority. If the owner key is compromised, an attacker could authorize a malicious issuer. Revocation is intentionally visible and does not erase history. The contract cannot determine whether a file is academically legitimate; it only manages the digest-to-issuer assertion made by an authorized account.

Consensus: Ethereum mainnet uses proof-of-stake validators to agree on canonical blocks and state. Remix VM and Ganache are local development environments; they do not demonstrate mainnet validator consensus.

7. Demonstration Output

The following is an expected Remix VM interaction transcript to reproduce. It is clearly illustrative and is not presented as a transaction captured from a deployed chain. Capture your own Remix screenshots after running the README steps if the submission requires screenshots.

```
Deploy CertiChain from account[0] success; account[0] is owner and issuer
Issue 0x7f...a91c from account[0] success; CertificateIssued event
Verify 0x7f...a91c exists=true, valid=true, issuer=account[0]
Revoke 0x7f...a91c from account[0] success; CertificateRevoked event
Verify 0x7f...a91c again exists=true, valid=false
Issue from unauthorized account reverted: caller is not authorized
```

Expected transaction behavior

Issuance and revocation are state-changing transactions. Each consumes gas and emits an event. Verification is a view call and can be executed without a transaction fee in Remix. Remix VM resets on environment reset; Ganache can be used when repeatable local chain state is preferred.

8. Run the Prototype

1. Open Remix IDE and create a Solidity file from contracts/CertiChain.sol.
2. Select compiler 0.8.24 or a compatible Solidity 0.8.x release and compile.
3. Choose Remix VM in Deploy & Run, select account[0], and deploy.
4. Use an off-chain SHA-256 digest of a sample file as bytes32 input to issueCertificate.
5. Query verifyCertificate from any account, revoke as issuer/owner, and query again.
6. To use Ganache, start a local chain and connect Remix to its RPC provider; record chain ID, address, and transaction hashes in the README after deployment.

The included helper scripts/hash_certificate.py prints a 32-byte SHA-256 fingerprint in 0x-prefixed form. Use a sample or synthetic document only. A production system must specify canonicalization because file metadata or encoding changes can alter the digest.

9. Product Assessment

CertiChain demonstrates a practical blockchain pattern: use a smart contract as a shared registry of issuer claims, preserve a transparent event trail, restrict writes to authorized wallets, and allow public verification. The demonstration highlights replicated state and transaction execution on an EVM chain while keeping document data outside the ledger.

The design has clear limits. A digest match proves file identity relative to a prior registration, not the truth of the certificate. Access to the source file is still necessary to compute a matching digest. Public chains create permanent metadata, and the registry owner introduces a governance trust assumption.

10. Roadmap

- Add a multisignature issuer-administration policy and issuer key rotation.
- Add issuer administration and network-aware controls to the browser interface.
- Support revocation reasons through privacy-safe controlled codes or a signed off-chain statement.
- Add independent contract review, automated security analysis, and deployment verification.
- Evaluate a permissioned consortium chain if institutions need controlled validator membership.

References and project files

Solidity documentation: https://docs.soliditylang.org/
Hardhat documentation: https://hardhat.org/docs/getting-started
Remix IDE: https://remix.ethereum.org/
Proiect source and run instructions: README.md and contracts/CertiChain.sol

Portfolio status: the interface and contract are prepared for local EVM demonstration. No public deployment, audited security review, or production use is claimed.